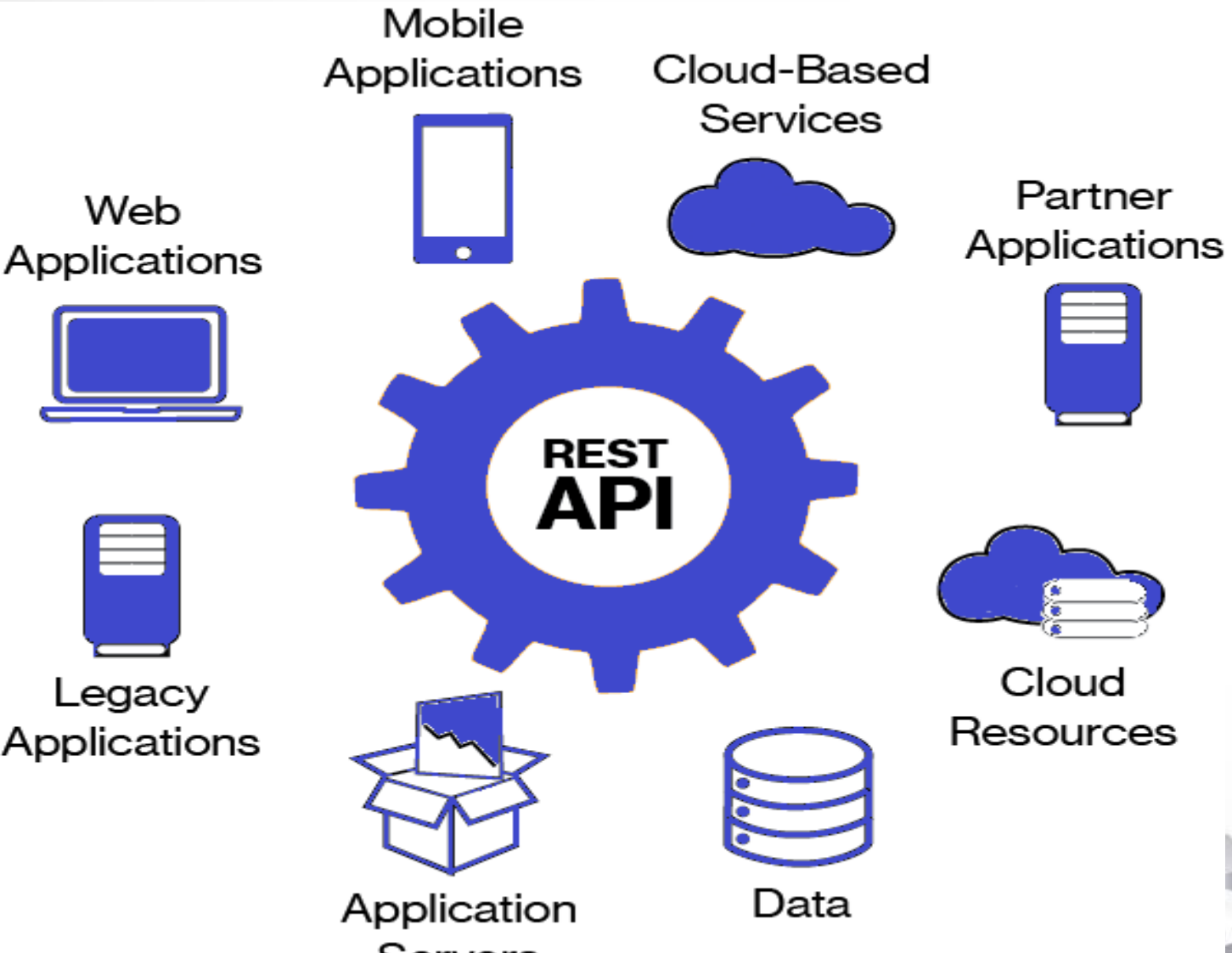
Web Architecture



Web architecture

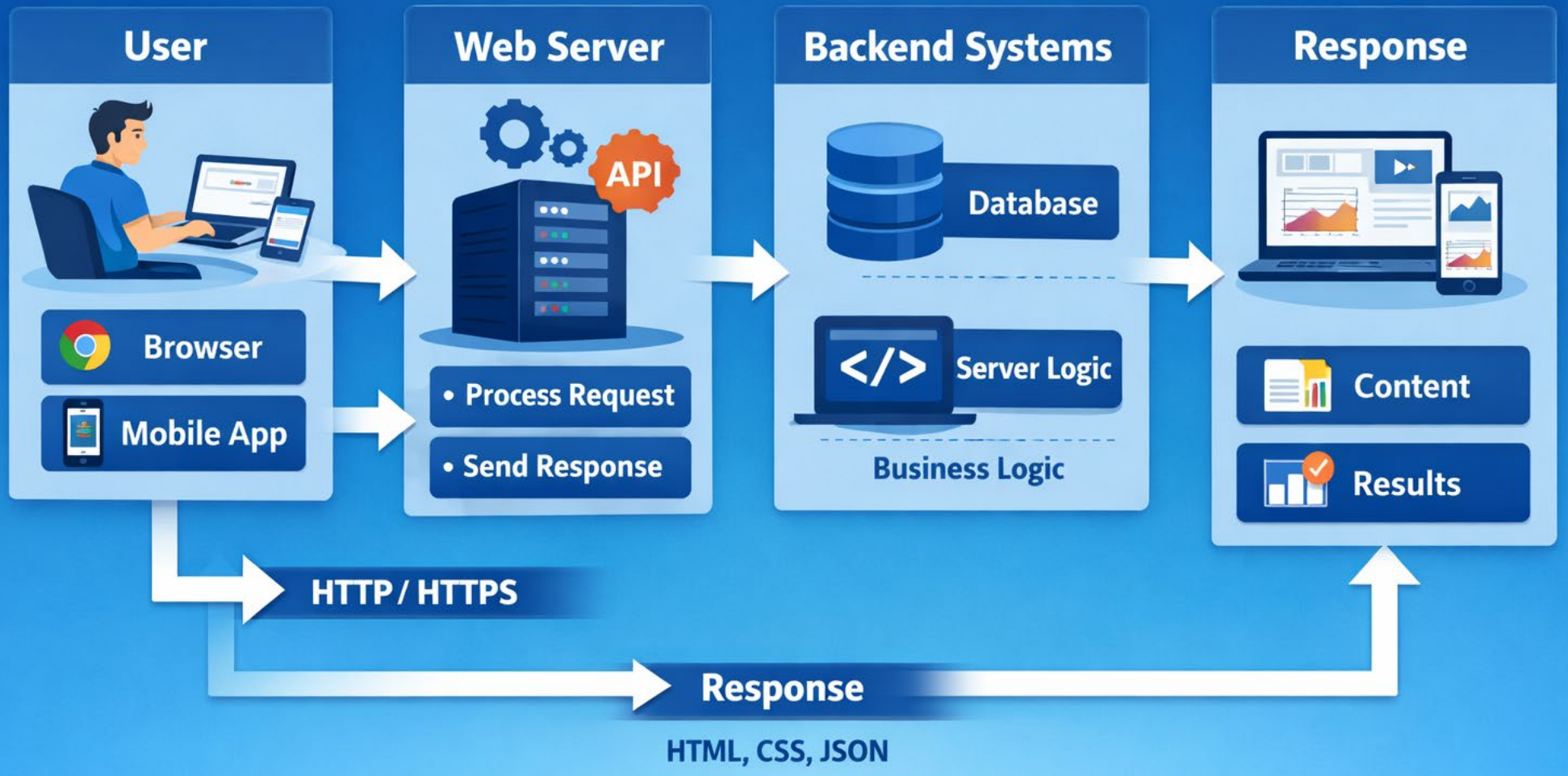


Web architecture - refers to the structural design of web applications, including both front-end and back-end components, as well as the communication protocols and technologies used to connect them.

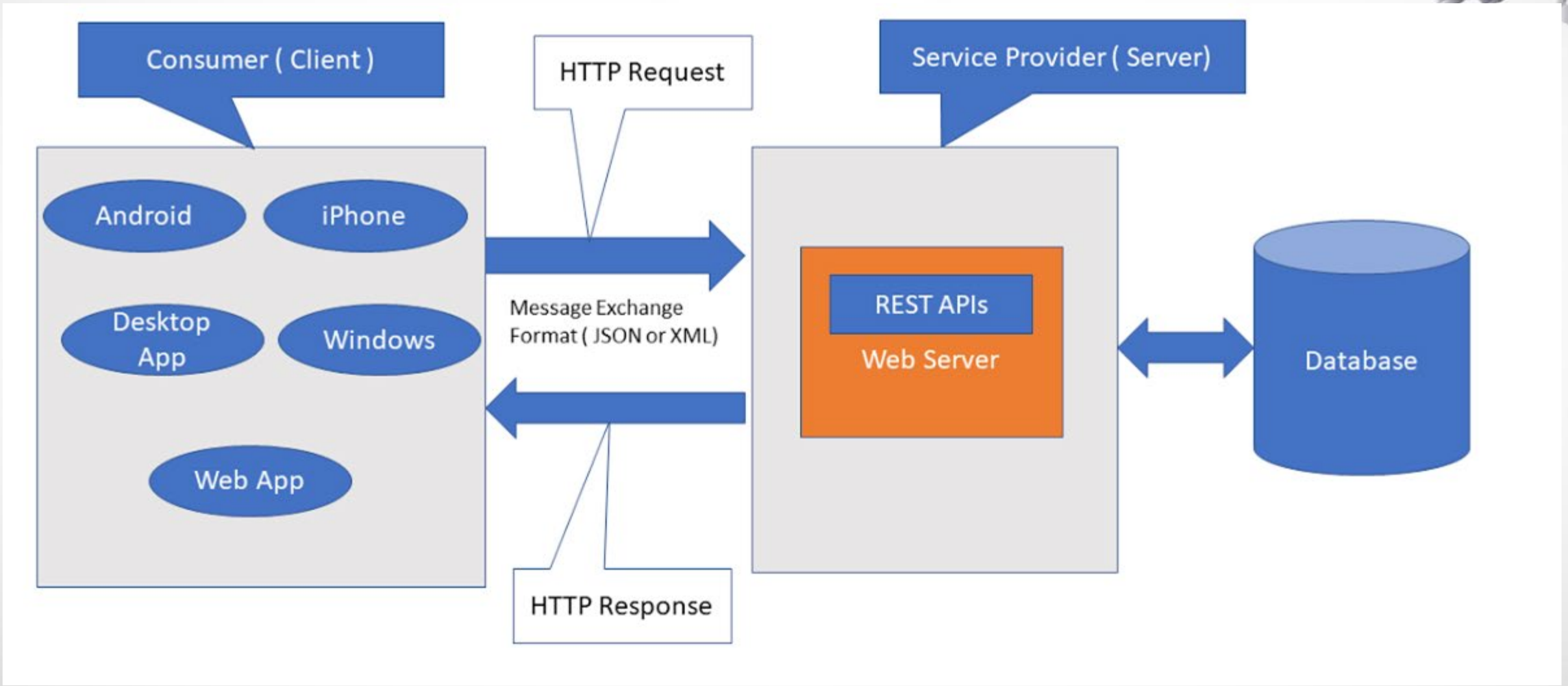
Structural design of web applications - a break down

1. Front-End Components
2. Back-End Components
3. Communication Protocols and related Technologies

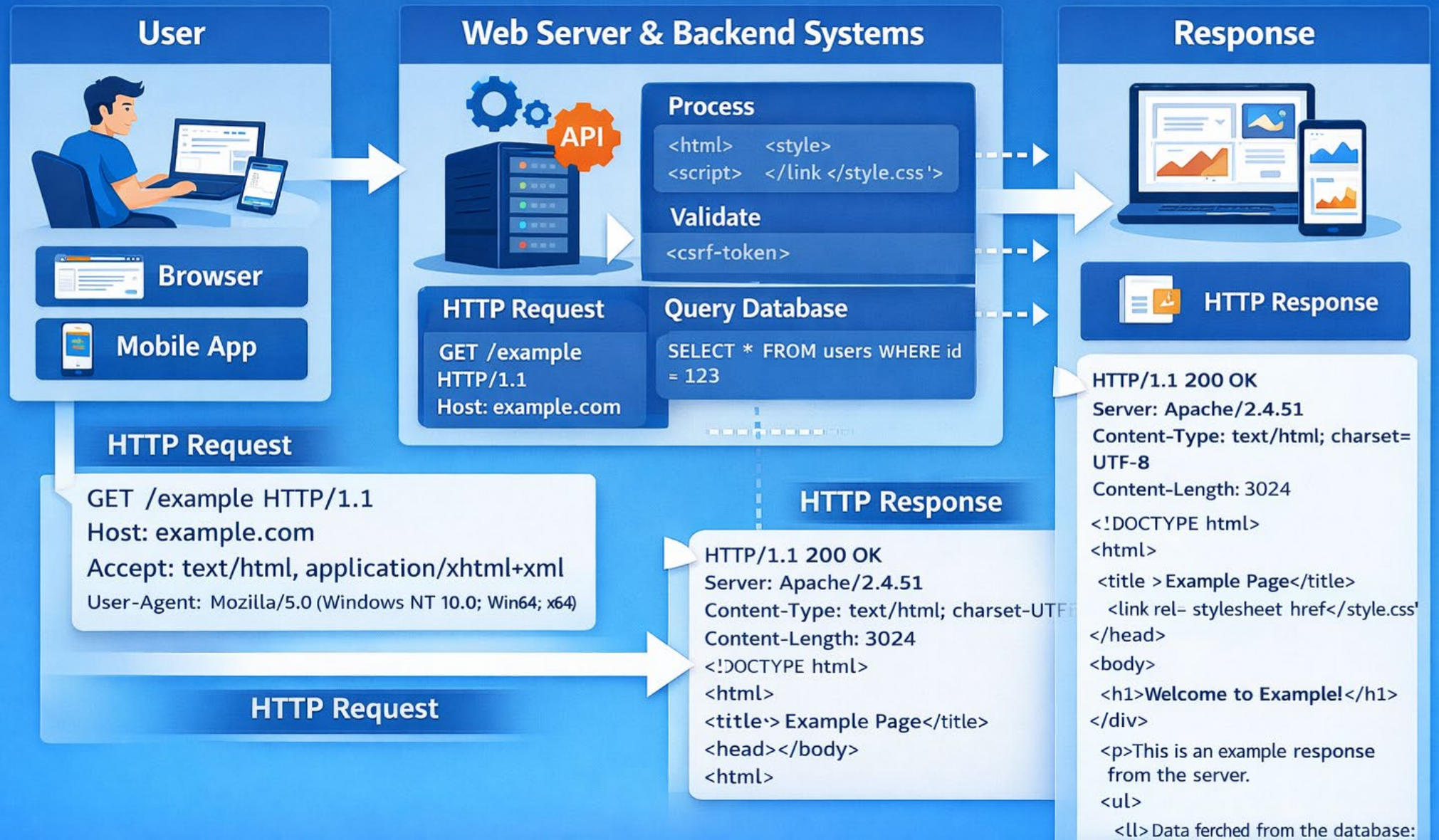
How a Web Application Works



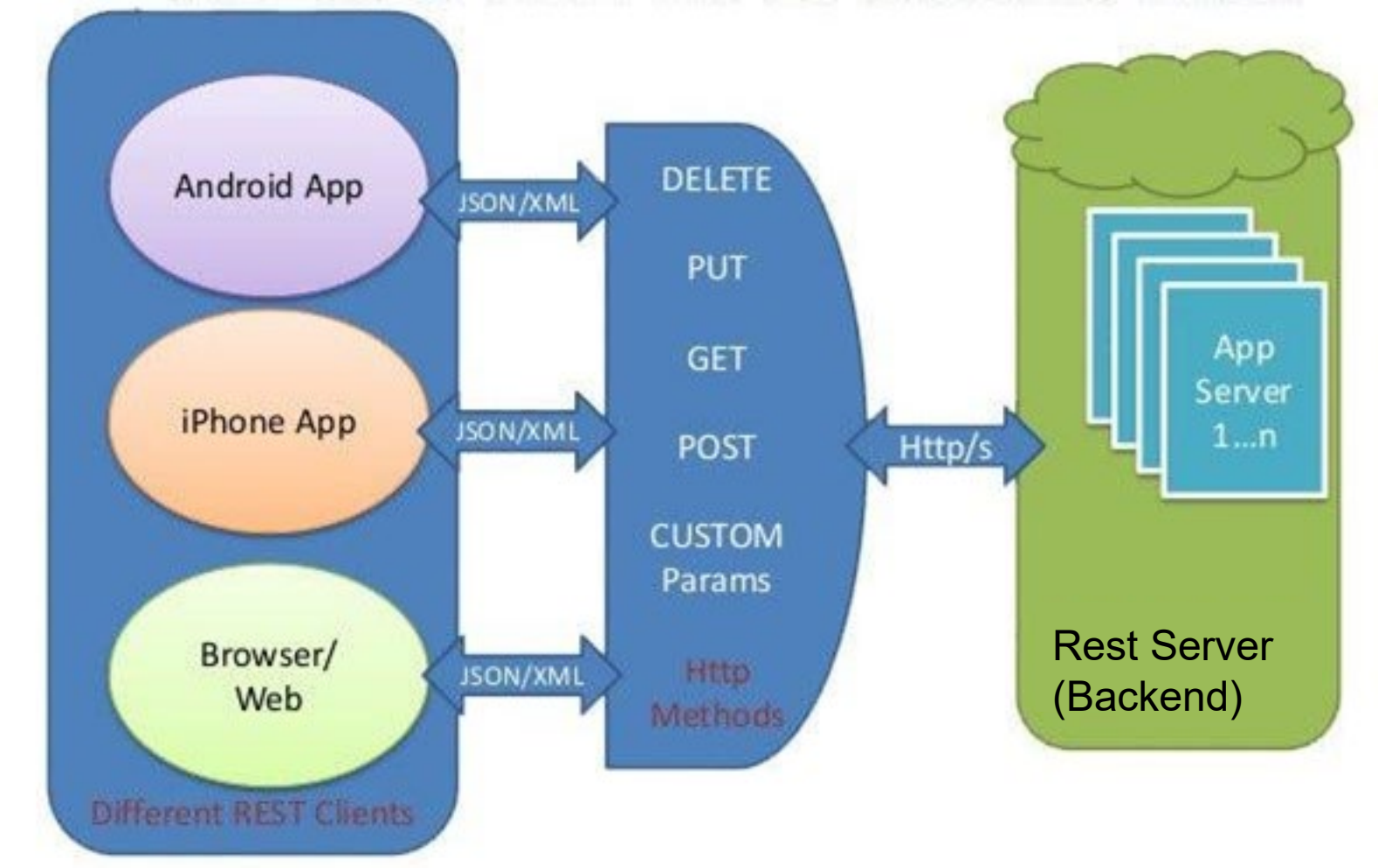
Web Architecture (Rest Based)



How HTML is Implemented for HTTP Request & HTTP Response



Web Architecture (Rest Based)



1. Front-End Components



1.1 HTML, CSS, and JavaScript

- HTML (Hypertext Markup Language): Provides the structure and content of web pages.
- CSS (Cascading Style Sheets): Defines the presentation and styling of HTML elements.
- JavaScript: Adds interactivity, dynamic behavior, and manipulation of HTML and CSS

1.2 Front-End Frameworks and Libraries

- Frameworks like React, Angular, and Vue.js provide structured ways to build complex user interfaces.
- Libraries such as jQuery offer utilities for DOM manipulation and AJAX requests

1. Front-End Components (cont..)



2.1 Browser Rendering Engine

- Plays a central role in the web browsing experience by parsing HTML, applying CSS styles, executing JavaScript, and ultimately rendering the final web page that users interact with in their browsers
- example:
  - Blink - developed by Google - used in Google Chrome , Microsoft Edge and Opera
  - WebKit - by Apple - used in Apple's Safari browser
  - Gecko - developed by Mozilla - used in the Firefox browser. Known for its standards compliance and rendering accuracy.
  - Trident (MSHTML) - by Microsoft - rendering engine used in older Internet Explorer browser
  - EdgeHTML - by Microsoft - originally used in Microsoft Edge browser (now replaced by Blink engine)

2. Back-End Components:



2.1 Server-Side Programming Languages

- Example: JavaScript (Node.js), Python (Django, Flask), Ruby (Ruby on Rails), Java (Spring Boot), and PHP power the back-end logic.
- They handle data processing, business logic, and interaction with databases

2.2 Web Servers:

- Software responsible for serving web content to clients.
- Examples include Apache, Nginx, and Microsoft IIS.

2. Back-End Components (cont...)



2.3 Databases

- Store and manage application data.
- example: relational databases (MySQL, PostgreSQL), NoSQL databases (MongoDB, Cassandra), and in-memory databases (Redis).

2.4 Server-Side Frameworks and APIs:

- Frameworks like Express.js (for Node.js), Django REST Framework (for Python), and Spring Boot (for Java) streamline back-end development.
- RESTful APIs provide standardized ways for clients to interact with server resources.

2.4 Web Services and APIs

- enable the communication and interaction between different software systems.
- provide a standardized way for applications to exchange data and functionality

Web Services: software systems designed to support interoperable machine-to-machine interaction over a network. typically follow standards such as SOAP (Simple Object Access Protocol) or REST (Representational State Transfer) for communication.

APIs: define a set of rules and protocols that allow different software applications to communicate with each other.

APIs can expose functionality or data from one application to another in a controlled and secure manner.

RESTful APIs: (Representational State Transfer) is an architectural style for designing networked applications.

based on REST principles and use HTTP methods (GET, POST, PUT, DELETE) to perform CRUD (Create, Read, Update, Delete) operations on resources.

3. Communication Protocols and Related Technologies

3.1 HTTP (Hypertext Transfer Protocol):

- The foundation of data communication on the web.
- Defines how messages are formatted and transmitted between clients and servers.

3.2 JSON (JavaScript Object Notation) and XML (eXtensible Markup Language):

- Common formats for representing structured data exchanged between clients and servers.
- JSON is lightweight, easy to parse, and widely used in modern web applications.
- XML is more verbose and used in certain legacy systems or specialized scenarios

3. Communication Protocols and Related Technologies (cont...)

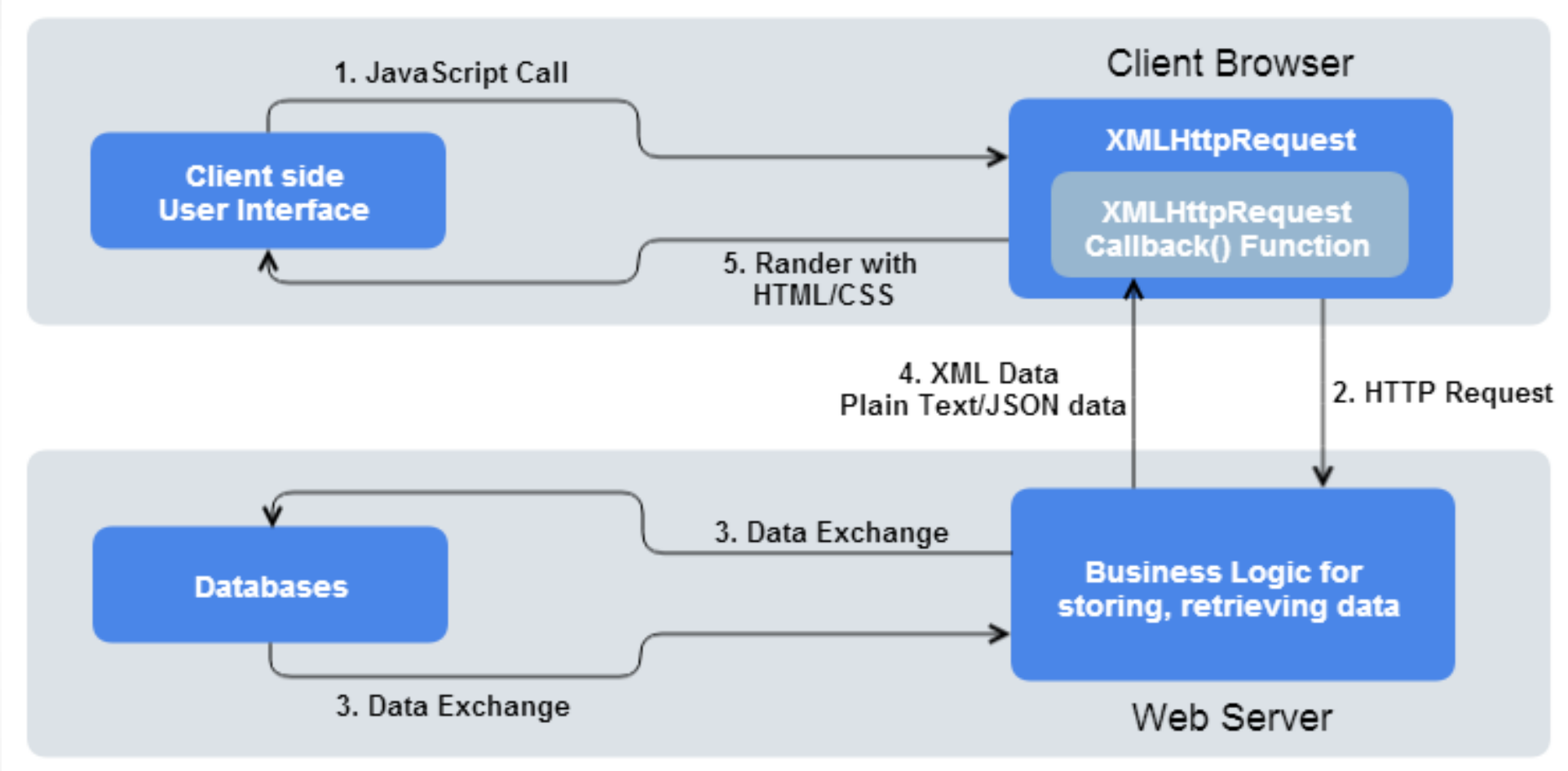


3.3 APIs (Application Programming Interfaces)

- Define rules and protocols for how different software components can interact and exchange data.
- RESTful APIs are commonly used in web applications, providing a stateless, uniform interface for accessing and manipulating resources

3.4 AJAX (Asynchronous JavaScript and XML):

- Enables asynchronous communication between the browser and server without refreshing the entire page.
- Utilizes the XMLHttpRequest (XHR) object or fetch API to make HTTP requests.



How Asynchronous Wab Application work using XHR, AJAX or Fetch API

3.4. The evolution of asynchronous communication (from XmlHttpRequest, Ajax, ... to WebSocket)



3.4.1 XmlHttpRequest

- Introduced by Microsoft in Internet Explorer 5, allowed JavaScript to make asynchronous HTTP requests to the server without reloading the entire page
- Became the foundation for AJAX - a technique that revolutionized web development
- Developers started using XHR to fetch XML, HTML, JSON, or any other data from the server asynchronously

3.4.2 jQuery AJAX

- a popular JavaScript library, abstracted away the complexity of XHR, providing a simplified interface for making asynchronous requests
- allowed developers to perform common AJAX operations with fewer lines of code and better cross-browser compatibility

3.4. The evolution of asynchronous communication (from XmlHttpRequest, Ajax, ... to WebSocket)



3.4.3 Fetch API

- introduced as part of the ECMAScript 6 (ES6) standard, provides a modern, promise-based interface for making HTTP requests in the browser
- offers a more flexible and powerful alternative to XHR, supporting features like streaming responses and a simpler syntax for handling requests and responses
- easier to work with asynchronous code and handle complex scenarios like chaining multiple requests

3.4.2 Async/Await

- introduced in ES8 that simplifies asynchronous programming in JavaScript
- using the **async** and **await** keywords, developers can write asynchronous code in a synchronous style, making it more readable and easier to reason about

3.4. The evolution of asynchronous communication (from XmlHttpRequest, Ajax, ... to WebSocket)



3.4.4 WebSockets

- While not strictly asynchronous communication over HTTP, WebSockets represent another significant evolution in web app communication
- provide full-duplex communication channels over a single, long-lived TCP connection, enabling real-time, bidirectional communication between the client and server
- allow servers to initiate communication with clients, making them ideal for applications requiring low-latency, real-time updates, such as chat apps, online gaming, and live data feeds

Summary



- The structural design of web applications involves front-end components (HTML, CSS, JavaScript), back-end components (server-side programming languages, databases, frameworks), and communication protocols/technologies (HTTP, AJAX, WebSockets, JSON/XML, APIs).
- These elements work together to create interactive, dynamic, and responsive web experiences, with data flowing seamlessly between the client and server.